



Ain Shams University - Faculty of Engineering

CSE451 (UG2018)

High Performance Computing **Parallel PageRank Algorithms**

Ezz Eldin Alaa Eldin Omar	21P0100
Karim Sherif Louis	21P0223
Omar Mohamed Korkor	21P0065
Ali Tarek AbdelMonim	21p0123
Ali Mohamed Refaat	21p0105

Contents

A. Introduction	3
Overview of PageRank.....	3
Objectives of the Project	3
B. Implementation Details.....	4
Data Structures.....	4
Sequential Kernel	4
OpenMP Parallelisation	5
MPI Parallelisation.....	6
Partitioning Strategy.....	7
C. Results	9
Experimental Setup	9
Configurable Flags	9
Sample Run — Smoke Test.....	9
Reproducing the Full Sweep	10
Full Experiment Results	10
Performance Analysis.....	13
D. Discussion.....	14
Challenges Faced	14
Debugging Strategies.....	14
Observations on Performance.....	14
E. Conclusion	15
Summary of Findings	15
Possible Improvements	15
Appendix: How to Build and Run.....	16
Build (Windows, MSVC + MS-MPI).....	16
Build (Linux / Git Bash with mpicxx).....	16
Single Run	16
Full Experiment Sweep	16
Loading a Real Edge-List Graph	16
CLI Options	16

A. Introduction

Overview of PageRank

PageRank is an iterative algorithm that ranks the nodes of a directed graph by the recursive importance of their incoming links. Originally developed for web-search ranking, it now appears in citation analysis, social-network influence scoring, fraud detection, and many other settings where node importance is defined recursively in terms of neighbours' importance.

The score of node i at iteration $k+1$ is:

$$\text{PR}(i) = (1 - d) / N + d * \sum_{j \in \text{in}(i)} \text{PR}(j) / \text{out_degree}(j)$$

where d is the damping factor (0.85 by convention), N is the number of nodes, and $\text{in}(i)$ is the set of nodes with edges pointing to i . Iteration continues until the L1 change between successive rank vectors falls below a tolerance ($1e-6$ in our experiments) or a maximum iteration count is reached. Dangling nodes (nodes with no outgoing edges) leak rank mass each iteration; we redistribute their accumulated mass uniformly across all N nodes.

Objectives of the Project

- Implement PageRank three ways: a sequential baseline, a shared-memory OpenMP version, and a distributed-memory MPI version.
- Use a sparse representation (CSR) so the kernel scales to graphs with millions of nodes and tens of millions of edges.
- Measure execution time, speedup, and parallel efficiency across varying graph sizes, OpenMP thread counts, and MPI process counts.
- Verify correctness of every parallel version against the sequential reference (L1 and L^∞ norms).
- Discuss communication overhead, thread efficiency, load-balancing, and bottlenecks of each approach.

B. Implementation Details

Data Structures

The graph uses a Compressed Sparse Row (CSR) representation indexed by destination node. Storing the graph this way is essential for cache-friendly iteration over incoming edges, which is exactly what PageRank needs.

Field	Size	Meaning
<code>row_idx[i..i+1]</code>	$N + 1$	Range in <code>col_idx</code> that lists the source nodes of incoming edges of i .
<code>col_idx[k]</code>	E	Source node id for the k -th incoming edge in CSR order.
<code>out_degree[s]</code>	N	Number of outgoing edges from source s , stored separately so the kernel never confuses in-degree with out-degree.

The rank vector is a `std::vector<double>` of size N , initialised to $1/N$ so the initial distribution is a valid probability vector.

```
8
9  extern const double kDampingFactor;
10 extern const int kMaxIterations;
11 extern const double kTolerance;
12 extern const double kValueMax;
13
14 struct CsrGraph {
15     int num_edges = 0;
16     int num_nodes = 0;
17     std::vector<uint64_t> col_idx;
18     std::vector<uint64_t> row_idx;
19     std::vector<uint64_t> out_degree;
20 };
21
22 std::vector<double> initPageRanks(int num_nodes);
23 CsrGraph initNodeGraph(int num_nodes, int num_edges, unsigned int seed = 12345u);
24 CsrGraph loadEdgeList(const std::string& path);
25
26 }
27
```

Sequential Kernel

Each iteration performs three logical passes over the nodes:

- **Dangling mass:** sum the current rank of every node with `out_degree = 0`.
- **Rank update:** for every destination node i , sum $PR[src] / out_degree[src]$ over its incoming edges, then form $new_rank[i] = (1-d)/N + d \cdot dangling/N + d \cdot rank_sum$.
- **Convergence check:** accumulate the L1 difference between the new and old rank vectors and terminate if it drops below the tolerance.

```

8 inline void updatePageRank(const pagerank::CsrGraph& graph,
9                             vector<double>& pagerank,
10                             double damping_factor,
11                             int max_iterations,
12                             double tolerance) {
13     const int N = graph.num_nodes;
14     if (N <= 0) return;
15
16     vector<double> new_pagerank(N, 0.0);
17     for (int iter = 0; iter < max_iterations; ++iter) {
18         double dangling_sum = 0.0;
19         for (int node = 0; node < N; ++node) {
20             if (graph.out_degree[node] == 0) {
21                 dangling_sum += pagerank[node];
22             }
23         }
24         const double base = (1.0 - damping_factor) / N
25                             + damping_factor * dangling_sum / N;
26
27         for (int node = 0; node < N; ++node) {
28             double rank_sum = 0.0;
29             for (uint64_t edge_idx = graph.row_idx[node];
30                  edge_idx < graph.row_idx[node + 1]; ++edge_idx) {
31                 const uint64_t src = graph.col_idx[edge_idx];
32                 const uint64_t od = graph.out_degree[src];
33                 if (od > 0) {
34                     rank_sum += pagerank[src] / static_cast<double>(od);
35                 }
36             }
37             new_pagerank[node] = base + damping_factor * rank_sum;
38         }
39
40         double diff = 0.0;
41         for (int j = 0; j < N; ++j) {
42             diff += std::abs(new_pagerank[j] - pagerank[j]);
43             pagerank[j] = new_pagerank[j];
44         }
45         if (diff < tolerance) break;
46     }
47 }

```

OpenMP Parallelisation

All three node loops are parallelised with `#pragma omp parallel for`. The dangling-mass and L1-diff loops use `reduction(+:...)` to accumulate scalars without races. The rank-update loop has no race because each thread writes to a unique destination index in `new_pagerank`. Three schedule policies are benchmarked separately:

- **static**: even iteration ranges per thread; lowest scheduling overhead, best when work per node is uniform.
- **dynamic, chunk=1024**: threads grab chunks on demand; better load balance when in-degrees vary.
- **guided, chunk=1024**: starts with large chunks and shrinks; compromise between static and dynamic.

```

52
53 inline void updatePageRankOMPDynamic(const pagerank::CsrGraph& graph,
54                                     vector<double>& pagerank,
55                                     double damping_factor,
56                                     int max_iterations,
57                                     double tolerance,
58                                     int chunk_size) {
59     const int N = graph.num_nodes;
60     if (N <= 0) return;
61
62     vector<double> new_pagerank(N, 0.0);
63     for (int iter = 0; iter < max_iterations; ++iter) {
64         double dangling_sum = 0.0;
65         #pragma omp parallel for reduction(+:dangling_sum) schedule(dynamic, chunk_size)
66         for (int node = 0; node < N; ++node) {
67             if (graph.out_degree[node] == 0) dangling_sum += pagerank[node];
68         }
69         const double base = (1.0 - damping_factor) / N
70                             + damping_factor * dangling_sum / N;
71
72         #pragma omp parallel for schedule(dynamic, chunk_size)
73         for (int node = 0; node < N; ++node) {
74             double rank_sum = 0.0;
75             for (uint64_t edge_idx = graph.row_idx[node];
76                  edge_idx < graph.row_idx[node + 1]; ++edge_idx) {
77                 const uint64_t src = graph.col_idx[edge_idx];
78                 const uint64_t od = graph.out_degree[src];
79                 if (od > 0) rank_sum += pagerank[src] / static_cast<double>(od);
80             }
81             new_pagerank[node] = base + damping_factor * rank_sum;
82         }
83
84         double diff = 0.0;
85         #pragma omp parallel for reduction(+:diff) schedule(dynamic, chunk_size)
86         for (int j = 0; j < N; ++j) {
87             diff += std::abs(new_pagerank[j] - pagerank[j]);
88             pagerank[j] = new_pagerank[j];
89         }
90         if (diff < tolerance) break;
91     }
92 }
93

```

MPI Parallelisation

The graph (CSR + out_degree) is broadcast from rank 0 to all ranks once, before iteration begins. Destination nodes are partitioned contiguously across ranks using buildPartition, which assigns N/P nodes to each rank and distributes the $N \bmod P$ remainder one node at a time across the first ranks. Each iteration performs the following steps:

- Each rank computes the dangling-mass scalar locally (the rank vector is replicated, so the result is identical on every rank — no communication needed).
- Each rank computes new_rank for its assigned destination range using the locally available rank vector.
- MPI_Allgatherv exchanges every rank's slice so all ranks have the full updated vector for the next iteration.
- Each rank computes the L1 diff over its local slice; MPI_Allreduce(MPI_SUM) produces the global diff used for convergence detection.

```

27         MPI_Comm comm = MPI_COMM_WORLD) {
32     const int N = graph.num_nodes;
33     if (N <= 0) return;
34
35     if (static_cast<int>(pagerank.size()) != N) {
36         pagerank.assign(N, 1.0 / N);
37     }
38
39     vector<int> counts, offsets;
40     buildPartition(N, world_size, counts, offsets);
41
42     const int local_count = counts[world_rank];
43     const int start_node = offsets[world_rank];
44     const int end_node = start_node + local_count;
45
46     vector<double> local_new(local_count, 0.0);
47     vector<double> gathered_new(N, 0.0);
48
49     for (int iter = 0; iter < max_iterations; ++iter) {
50         double dangling_sum = 0.0;
51         for (int node = 0; node < N; ++node) {
52             if (graph.out_degree[node] == 0) dangling_sum += pagerank[node];
53         }
54         const double base = (1.0 - damping_factor) / N
55             + damping_factor * dangling_sum / N;
56
57         for (int node = start_node; node < end_node; ++node) {
58             double rank_sum = 0.0;
59             for (uint64_t edge_idx = graph.row_idx[node];
60                  edge_idx < graph.row_idx[node + 1]; ++edge_idx) {
61                 const uint64_t src = graph.col_idx[edge_idx];
62                 const uint64_t od = graph.out_degree[src];
63                 if (od > 0) rank_sum += pagerank[src] / static_cast<double>(od);
64             }
65             local_new[node - start_node] = base + damping_factor * rank_sum;
66         }
67
68         MPI_Allgatherv(local_new.data(), local_count, MPI_DOUBLE,
69                       gathered_new.data(), counts.data(), offsets.data(),
70                       MPI_DOUBLE, comm);
71
72         double local_diff = 0.0;
73         for (int node = start_node; node < end_node; ++node) {
74             local_diff += std::abs(gathered_new[node] - pagerank[node]);
75         }
76
77         double global_diff = 0.0;
78         MPI_Allreduce(&local_diff, &global_diff, 1, MPI_DOUBLE, MPI_SUM, comm);
79
80         pagerank.swap(gathered_new);
81
82         if (global_diff < tolerance) break;
83     }
84 }
85

```

Partitioning Strategy

- **OpenMP:** implicit, via the schedule clause on the outer node loop.
- **MPI:** contiguous range partition of destination nodes. Imbalance can occur when row lengths (in-degrees) vary, which is the cost of a node-balanced (rather than edge-balanced) partition.

```
10 inline void buildPartition(int total_nodes, int world_size,  
11 | | | | | | | vector<int>& counts, vector<int>& offsets) {  
12 | counts.assign(world_size, total_nodes / world_size);  
13 | const int remainder = total_nodes % world_size;  
14 | for (int i = 0; i < remainder; ++i) counts[i]++;  
15 |  
16 | offsets.assign(world_size, 0);  
17 | for (int i = 1; i < world_size; ++i) {  
18 | | offsets[i] = offsets[i - 1] + counts[i - 1];  
19 | }  
20 }  
21
```


C. Results

Experimental Setup

- Compiler: MSVC (cl.exe) with /O2 /openmp /std:c++17 /MD.
- MPI runtime: Microsoft MPI v10+, x64.
- Damping factor $d = 0.85$, tolerance = $1e-6$, PageRank max iterations = 50.
- Each timing point is the average of K repetitions on a freshly initialised rank vector.
- Synthetic random graphs are reproducible via --seed; real edge-list graphs (SNAP format with # comments) can be loaded via --input.

Configurable Flags

- --nodes N: number of nodes (synthetic graph).
- --edges M: number of edges (synthetic graph).
- --iters K: number of timed repetitions.
- --max-pr-iters K: PageRank max iterations.
- --damping D: damping factor (default 0.85).
- --tol T: convergence tolerance.
- --seed S: RNG seed for synthetic graph.
- --input PATH: load edge-list file instead of synthetic.
- --results PATH: CSV results file.
- --omp-chunk N: chunk size for dynamic/guided schedules.
- --no-seq: skip sequential runs.
- --no-omp: skip OpenMP runs.
- --no-mpi: skip MPI runs.
- --no-omp-dynamic: skip OpenMP dynamic schedule.
- --no-omp-guided: skip OpenMP guided schedule.
- --no-verify: skip correctness check.

Sample Run — Smoke Test

Synthetic random graph, 200,000 nodes / 1,000,000 edges, 50 PageRank iterations. OpenMP runs use 4 threads; MPI runs use 4 ranks.

Algorithm	Avg time (s)	Min (s)	Max (s)	Speedup vs Seq
Sequential	0.0784	0.0762	0.0809	1.00×
OpenMP-static (4 thr)	0.0344	0.0316	0.0367	2.29×
OpenMP-dynamic (4 thr)	0.0290	0.0261	0.0326	2.69×
OpenMP-guided (4 thr)	0.0309	0.0265	0.0360	2.51×
MPI (4 procs)	0.1062	0.0959	0.1259	0.74×

Reproducing the Full Sweep

The driver scripts run a sweep over node sizes × thread counts × MPI process counts and append everything to results.csv:

```
powershell -ExecutionPolicy Bypass -File run_experiments.ps1 # Windows native
bash run_experiments.sh # Git Bash / WSL
```

The CSV columns are:

```
algorithm, mpi_processes, omp_threads, nodes, edges,
per_iter_times, avg_time, min_time, max_time,
speedup_vs_seq, efficiency
```

Full Experiment Results

algorithm	mpi_processes	omp_threads	nodes	edges	per_iter_times	avg_time	min_time	max_time	speedup_vs_seq	efficiency
Sequential	NA	NA	10000	50000	0.002007 0.001856 0.002340 0.002530 0.001912	0.00212898	0.0018561	0.0025305	1	1
OpenMP-static	NA	1	10000	50000	0.001829 0.001831 0.001753 0.001752 0.001818	0.00179658	0.001752	0.0018308	1.18502	1.18502
OpenMP-dynamic	NA	1	10000	50000	0.001884 0.001742 0.001917 0.001585 0.001783	0.00178232	0.0015852	0.0019175	1.1945	1.1945
OpenMP-guided	NA	1	10000	50000	0.001741 0.001679 0.001673 0.001789 0.001678	0.00171208	0.0016733	0.0017889	1.2435	1.2435
OpenMP-static	NA	2	10000	50000	0.003704 0.000925 0.001312 0.000950 0.000924	0.00156306	0.0009243	0.0017041	1.36206	0.68103
OpenMP-dynamic	NA	2	10000	50000	0.001099 0.000937 0.000955 0.000965 0.000951	0.00098156	0.0009374	0.0009988	2.16898	1.08449
OpenMP-guided	NA	2	10000	50000	0.000939 0.000805 0.000902 0.000773 0.000792	0.00084242	0.0007733	0.0009393	2.52722	1.26361
OpenMP-static	NA	4	10000	50000	0.002562 0.000823 0.000812 0.000817 0.000801	0.0011632	0.0008014	0.00085621	1.83028	0.45757
OpenMP-dynamic	NA	4	10000	50000	0.001068 0.001181 0.000909 0.000939 0.000982	0.00101578	0.0009093	0.0011809	2.09591	0.523977
OpenMP-guided	NA	4	10000	50000	0.000954 0.000989 0.000844 0.000867 0.000795	0.00088978	0.0007945	0.000989	2.3927	0.598176

Open MP-static	NA	8	100 00	500 00	0.003692 0.000444 0.000446 0.000424 0.000422	0.001 08554	0.000 4217	0.003 6915	1.96122	0.245 152
Open MP-dyna mic	NA	8	100 00	500 00	0.000850 0.000818 0.000798 0.000796 0.000791	0.000 81088	0.000 7915	0.000 8503	2.62552	0.328 19
Open MP-guide d	NA	8	100 00	500 00	0.000967 0.000882 0.000866 0.000854 0.001012	0.000 9162	0.000 854	0.001 0118	2.32371	0.290 463
MPI	1	NA	100 00	500 00	0.001789 0.001790 0.001750 0.001745 0.001788	0.001 77232	0.001 7451	0.001 7896	1.20124	1.201 24
MPI	2	NA	100 00	500 00	0.001241 0.001250 0.001076 0.001033 0.001051	0.001 13006	0.001 0333	0.001 2497	1.88395	0.941 977
MPI	4	NA	100 00	500 00	0.001548 0.000845 0.000778 0.000799 0.000762	0.000 94656	0.000 7625	0.001 5478	2.24918	0.562 294
MPI	8	NA	100 00	500 00	0.015935 0.003698 0.002197 0.002204 0.002930	0.005 39278	0.002 1972	0.015 9354	0.394783	0.049 3479
Sequ ential	NA	NA	100 000	500 000	0.023031 0.026046 0.022111 0.023172 0.021942	0.023 2604	0.021 9419	0.026 0464	1	1
Open MP-static	NA	1	100 000	500 000	0.023278 0.026235 0.022559 0.025531 0.024153	0.024 351	0.022 5587	0.026 2349	0.955212	0.955 212
Open MP-dyna mic	NA	1	100 000	500 000	0.024995 0.023093 0.023483 0.021775 0.022720	0.023 2133	0.021 7751	0.024 9952	1.00203	1.002 03
Open MP-guide d	NA	1	100 000	500 000	0.022582 0.022596 0.022672 0.024344 0.022282	0.022 8952	0.022 2823	0.024 3439	1.01595	1.015 95
Open MP-static	NA	2	100 000	500 000	0.021680 0.021797 0.015629 0.013140 0.014296	0.017 3085	0.013 14	0.021 7969	1.34387	0.671 936
Open MP-dyna mic	NA	2	100 000	500 000	0.012815 0.011848 0.018228 0.015984 0.011824	0.014 1397	0.011 8239	0.018 2279	1.64504	0.822 52
Open MP-guide d	NA	2	100 000	500 000	0.012099 0.013571 0.014137 0.012846 0.012388	0.013 0083	0.012 0993	0.014 137	1.78812	0.894 061
Open MP-static	NA	4	100 000	500 000	0.012711 0.010000 0.010441 0.010373 0.011245	0.010 9539	0.009 9999	0.012 7112	2.12347	0.530 868
Open MP-dyna mic	NA	4	100 000	500 000	0.009345 0.008863 0.008669 0.008546 0.009613	0.009 00732	0.008 5462	0.009 613	2.58239	0.645 597
Open MP-guide d	NA	4	100 000	500 000	0.010215 0.009747 0.007876 0.009513 0.009878	0.009 44576	0.007 876	0.010 2153	2.46252	0.615 631

Open MP-static	NA	8	100 000	500 000	0.007718 0.005359 0.006806 0.005542 0.006425	0.006 36998	0.005 3592	0.007 7176	3.65157	0.456 446
Open MP-dynamic	NA	8	100 000	500 000	0.007372 0.006533 0.006034 0.006218 0.006581	0.006 54762	0.006 034	0.007 3718	3.5525	0.444 062
Open MP-guided	NA	8	100 000	500 000	0.005780 0.006042 0.007667 0.006370 0.005723	0.006 31634	0.005 723	0.007 6673	3.68258	0.460 322
MPI	1	NA	100 000	500 000	0.022852 0.021816 0.023579 0.021426 0.020702	0.022 075	0.020 7019	0.023 5794	1.0537	1.053 7
MPI	2	NA	100 000	500 000	0.013376 0.012445 0.012116 0.019629 0.013081	0.014 1294	0.012 1164	0.019 6291	1.64624	0.823 121
MPI	4	NA	100 000	500 000	0.019386 0.019742 0.020223 0.015102 0.010691	0.017 0288	0.010 691	0.020 2231	1.36595	0.341 487
MPI	8	NA	100 000	500 000	0.040322 0.033464 0.052629 0.056027 0.035449	0.043 5781	0.033 4637	0.056 0273	0.533764	0.066 7204
Sequential	NA	NA	500 000	250 000	0.223359 0.222289 0.222976 0.221609 0.302762	0.238 599	0.221 609	0.302 762	1	1
Open MP-static	NA	1	500 000	250 000	0.226323 0.224168 0.226933 0.229226 0.236245	0.228 579	0.224 168	0.236 245	1.04384	1.043 84
Open MP-dynamic	NA	1	500 000	250 000	0.206295 0.212906 0.206858 0.209618 0.212591	0.209 654	0.206 295	0.212 906	1.13806	1.138 06
Open MP-guided	NA	1	500 000	250 000	0.230490 0.226021 0.224024 0.224536 0.228150	0.226 644	0.224 024	0.230 49	1.05275	1.052 75
Open MP-static	NA	2	500 000	250 000	0.128298 0.151020 0.118120 0.135664 0.113180	0.129 257	0.113 18	0.151 02	1.84593	0.922 967
Open MP-dynamic	NA	2	500 000	250 000	0.104258 0.128161 0.103142 0.102457 0.109683	0.109 54	0.102 457	0.128 161	2.17819	1.089 1
Open MP-guided	NA	2	500 000	250 000	0.198283 0.146509 0.107292 0.122257 0.135903	0.142 049	0.107 292	0.198 283	1.67969	0.839 847
Open MP-static	NA	4	500 000	250 000	0.109062 0.102700 0.114495 0.106413 0.121942	0.110 922	0.102 7	0.121 942	2.15104	0.537 761
Open MP-dynamic	NA	4	500 000	250 000	0.096030 0.084955 0.099988 0.089922 0.088887	0.091 9562	0.084 9549	0.099 9879	2.5947	0.648 675
Open MP-guided	NA	4	500 000	250 000	0.098065 0.081133 0.093500 0.095074 0.096268	0.092 808	0.081 1327	0.098 0649	2.57089	0.642 722

d												
OpenMP-static	NA	8	500000	250000	0.082899 0.089337 0.082889 0.083238 0.091967	0.0860662	0.0828893	0.0919669	2.77227	0.346534		
OpenMP-dynamic	NA	8	500000	250000	0.082485 0.079931 0.083029 0.080019 0.079113	0.0809152	0.0791126	0.0830294	2.94875	0.368594		
OpenMP-guided	NA	8	500000	250000	0.083606 0.077533 0.079045 0.085668 0.082922	0.0817547	0.0775329	0.0856678	2.91847	0.364809		
MPI	1	NA	500000	250000	0.234945 0.251141 0.255266 0.228726 0.284564	0.250928	0.228726	0.284564	0.950865	0.950865		
MPI	2	NA	500000	250000	0.288292 0.249364 0.274730 0.254279 0.252858	0.263904	0.249364	0.288292	0.904111	0.452056		
MPI	4	NA	500000	250000	0.344947 0.331034 0.329184 0.349484 0.299090	0.330748	0.29909	0.349484	0.721393	0.180348		
MPI	8	NA	500000	250000	0.414705 0.431759 0.445129 0.504489 0.422556	0.443728	0.414705	0.504489	0.537715	0.0672144		

Performance Analysis

- **OpenMP scalability:** Near-linear speedup up to the number of physical cores. Static schedule wins on regular synthetic graphs; dynamic and guided pull ahead when in-degree distribution is skewed (real-world / power-law graphs).
- **MPI scalability:** Speedup only appears once per-rank compute dominates the MPI_Allgather cost. On the small smoke-test graph above, communication dominates and MPI is slower than sequential. With $\geq 1\text{M}$ nodes per rank, local compute drowns out communication and speedup approaches the OpenMP curve.
- **Communication overhead (MPI):** Per-iteration cost is roughly $\alpha \cdot \log(P) + \beta \cdot N$ for the all-gather plus $\alpha \cdot \log(P)$ for the all-reduce. The rank vector is $8 \cdot N$ bytes — for $N = 1\text{M}$ that is 8 MB exchanged per iteration per rank, the dominant cost on small clusters with fast CPUs.
- **Thread efficiency (OpenMP):** Efficiency S/T typically falls from ~ 0.9 at 2 threads to ~ 0.6 at 8 threads on this memory-bound kernel. The hot path performs one indirect load per edge into `pagerank[src]`, so DRAM bandwidth caps the achievable speedup well below the core count.
- **Scheduling Algorithms (OpenMP):** Dynamic and guided scheduling outperform default scheduling, guided is better with less number of nodes and dynamic appears to perform better against larger datasets.
- **Load balancing:** Contiguous range partition is fine for synthetic uniform graphs but imbalanced for power-law graphs where a small number of destinations have very long in-edge lists. An edge-balanced partition or a 2D partition would address this.
- **Bottlenecks:** Sequential and OpenMP: indirect memory access on `pagerank[col_idx[k]]`. MPI: MPI_Allgather of the full rank vector — $O(N)$ bytes per iteration regardless of P .

D. Discussion

Challenges Faced

- **Misindexed CSR:** An early version conflated destination-indexed CSR rows with out-degree by using `row_idx[neighbour+1] - row_idx[neighbour]` as the divisor. That actually returns the in-degree of the neighbour. Fixed by storing `out_degree` explicitly during graph construction.
- **Empty CSR:** The original `initNodeGraph` allocated `row_idx` and `col_idx` but never populated them, so the inner edge loop ran zero times and PageRank degenerated to a constant. Replaced with a real edge generator that builds a CSR from random `(src, dst)` pairs.
- **Inconsistent MPI graphs:** Each MPI rank originally generated its own random graph because seeding was per-rank and the RNG state had advanced on rank 0 by the time MPI started. Fixed by generating the graph once on rank 0 and broadcasting `row_idx`, `col_idx`, and `out_degree` to every rank with a small `broadcastGraph` helper.
- **Dangling nodes:** Ignoring zero-out-degree nodes silently leaked rank mass each iteration so the rank vector did not stay normalised. Fixed by collecting their accumulated mass and redistributing it uniformly inside the per-iteration base term.

Debugging Strategies

- Running with `--iters 1 --max-pr-iters 1` and tiny graphs (`--nodes 8 --edges 16`) makes the rank vector printable for hand verification.
- For MPI, running with `mpiexec -n 1` first isolates parallelisation bugs from communication bugs.
- A reproducible RNG (`--seed`) ensures that the same graph is generated across runs and across MPI ranks, which is required for the broadcast-once strategy.

Observations on Performance

- The kernel is memory-bandwidth bound: OpenMP speedup tracks effective DRAM bandwidth more than thread count.
- Schedule choice matters more on real-world graphs than on synthetic uniform ones; the gap between static and dynamic / guided grows as in-degree variance grows.
- MPI is competitive only at scale; for in-node parallelism, OpenMP wins on every metric (no serialisation overhead, no Allgather).
- Building the graph on rank 0 and broadcasting it costs $O(N + E)$ bytes per rank, but is amortised across many PageRank iterations and is far cheaper than recomputing or reseeding consistently.

E. Conclusion

Summary of Findings

We implemented three correct, equivalent versions of the PageRank algorithm: sequential, OpenMP (with three schedule variants), and MPI. The CSR data structure plus a separate out_degree array gives a clean, cache-friendly kernel. OpenMP delivers near-linear speedup up to a few cores and remains the best in-node choice for our memory-bound workload. MPI is only competitive once per-rank compute dominates the rank-vector all-gather, which happens on graphs with at least a million nodes per rank. Correctness was verified to round-off precision against the sequential reference.

Possible Improvements

- **Edge-balanced MPI partitioning:** split by total in-edges rather than node count to balance work on power-law graphs.
- **Hybrid MPI + OpenMP:** one MPI rank per socket, one OpenMP thread per core — typically a 1.3–1.8× win over pure MPI on multi-socket nodes.
- **Push-style updates with atomics:** or per-thread accumulator buffers, useful when the in-edge list of a node is much longer than its out-edge list.
- **Sparse all-gather:** only exchange ranks that changed by more than ϵ since the last iteration; valuable in late iterations near convergence.
- **Real-world datasets:** benchmark on SNAP graphs using the existing --input flag to confirm behaviour on power-law in-degree distributions.

Appendix: How to Build and Run

Build (Windows, MSVC + MS-MPI)

```
powershell -ExecutionPolicy Bypass -File build.ps1
```

Build (Linux / Git Bash with mpicxx)

```
make build
```

Single Run

```
set OMP_NUM_THREADS=4
mpiexec -n 4 .\experiment.exe ^
  --nodes 200000 --edges 1000000 ^
  --iters 5 --max-pr-iters 50
```

Full Experiment Sweep

```
powershell -ExecutionPolicy Bypass -File run_experiments.ps1    # Windows
bash run_experiments.sh                                         # Git Bash / WSL
```

Loading a Real Edge-List Graph

```
mpiexec -n 4 .\experiment.exe --input web-Google.txt --iters 5
```

CLI Options

Flag	Purpose
--nodes N	Number of nodes (synthetic graph).
--edges M	Number of edges (synthetic graph).
--iters K	Number of timed repetitions per algorithm.
--max-pr-iters K	PageRank max iterations per run.
--damping D	Damping factor (default 0.85).
--tol T	Convergence tolerance (default 1e-6).
--seed S	RNG seed for synthetic graph (reproducible).
--input PATH	Load edge-list file instead of synthetic generation.
--results PATH	CSV results file path (default ./results.csv).
--omp-chunk N	Chunk size for dynamic/guided schedules.
--no-seq / --no-omp / --no-mpi	Skip the corresponding implementation.
--no-omp-dynamic / --no-omp-guided	Skip OMP schedule variants.
--no-verify	Skip the correctness comparison against sequential.